

Module 9 Lab:

PITest Mutation Tool – Syntax Based Testing

JR Clark (002035589)

NEU CSYE6255 (Summer 2026)

Professor Omar Badreldin / TA Karthik Kashyap

PART 1: Introduction & Technical Implementation

As noted in our instructions and class connect discussion, code may pass normal JUnit tests but still contain weaknesses that can be uncovered with mutation testing. PITest creates small temporary changes in the code – if the JUnit tests do not catch those mutations, that is an indication that the tests need to be improved to verify the correct outcomes, not just the correct execution.

This lab required implementing or verifying Java, Maven, IntelliJ IDEA, creating a clean Maven project using the sample code provided. All of these tools were previously installed and confirmed (see Appendix 1). Claude.ai was used to extract the sample code from the files.

The sample project, PITestPaymentRiskDemo, models a small payment risk service that receives a payment request and returns one of three outcomes: approved, rejected, or sent to manual review. It is a well-suited choice for this exercise because it goes beyond a simple calculator example. It includes input validation, conditional business rules, arithmetic calculations, numeric thresholds, an external dependency (fraud scoring), and a side effect (audit logging), giving PITest a realistic range of mutation targets across all five operator categories (conditional, return value, method call, arithmetic, incremental/specialized).

PART 2: Testing – JUnit & PITest – Round 1

Running the JUnit tests with the baseline sample code resulted in identical outcomes to the sample code and expected outcomes (see Appendix 2). The PITest results also matched the guide's own demo figures closely: 91% line coverage (51/56), 37% mutation coverage (39/106 killed) and 39% test strength. The real revelation is in viewing the JUnit and PITest results side by side. JUnit's clean pass told me the code worked as written; PITest's much lower mutation

score told me the tests weren't actually confirming that. The tests were touching almost all of the code without verifying that it was doing the right thing.

I used Claude.ai to examine the full PITest report (Appendix 2) and break down what was actually surviving, by operator category. The 106 mutations came from all five required categories, each targeting a different kind of mistake a developer could realistically make. Conditional mutation shifts or negates comparisons — like the fraud-score and amount thresholds on line 54 — to see if a test notices the boundary moved. Return value mutation swaps out what a method hands back, such as replacing a result with null or zero, to check whether anything downstream would catch a wrong answer. Method-call mutation removes a call entirely — the audit log entries on lines 32, 55, and 64 are examples — to see if a test would notice a side effect silently disappearing. Arithmetic mutation swaps operators, like turning addition into subtraction in the subtotal and discount calculations on lines 39, 94, and 104, to see if a test checks the actual number instead of just the outcome category. And incremental/specialized constant mutation alters loop counters and the business-rule constants themselves (the 0.85 fraud threshold, the \$2,500 manual-review limit, the \$100 free-shipping cutoff) to see if a test would notice if the underlying rule quietly changed.

That analysis traced the gap to its root cause: five of the six baseline tests asserted only `decision.status()`, leaving uncovered branches untested, audit trail messages checked for whether they were called but not with what arguments, inconsistent arithmetic left unverified, no boundary-value cases, and mock interactions unverified beyond their return values. The recommendations were specific and actionable, with the LLM estimating that the following fixes would knock out the majority of surviving mutants (see Table 1, below):

TABLE 1: LLM Testing Priority Order:

1. Cover the untested `authorizedAmount <= 0` rejection branch and the first-purchase discount line — these are **completely untested**, not just weakly tested.
2. Add exact-value assertions (not just status) on `authorizedAmount`, `rewardPoints`, and audit messages.
3. Add boundary-exact test cases for every threshold constant.
4. Verify mock call arguments (`customerId`, `country`) rather than just mocking a fixed return value.
5. Add direct unit tests for the four helper methods to isolate their internal math from `authorize()`'s end-to-end flow.

SOURCE: Claude.ai <https://claude.ai/share/0ff11bbc-8926-404e-8df6-593a2a7ce357>

Claude.ai analysis resulted in feedback that lines 42-43 are “dead code” and unreachable by legitimate tests. As a program manager, I would flag this kind of issue with my team as part of a discussion to determine impact on fixing this on the rest of the codebase. See Appendix 3 for full summary of recommended changes from Claude.ai. I decided to leverage my Claude.ai session to update the tests and re-run PITest to see what would happen.

PART 3: Testing – JUnit & PITest – Round 2

After importing the updated tests created with Claude.ai, the mutation coverage increased from 37% (39/106 killed) to 94% killed (100/106) (see Figure 14 in Appendix 4). The test improvements resulted in 95% line coverage, 94% mutation coverage and test strength of 98% leaving just a few mutations which are all related to the unreachable lines of code at 42-43 (for full report, see Appendix 4). According to Claude – this is “defensive/dead code” rather than a test-coverage gap, and our final score represents effectively complete coverage of all reachable logic.

It’s important to note that just a handful of well prioritized tests did most of the work – some were improved versions of the original six – and closed nearly every gap from Round 1.

The biggest improvement was in checking actual numbers instead of simply checking the status.

The same six mutations left from Round 1 were all tied to the same branch that probably shouldn't exist. As a program manager, that's exactly the kind of finding I'd bring back to a dev team: not "go write more tests," but "let's talk about whether this code should be here at all."

PART 4: Key Takeaways

As a program manager, I have always depended on the team's Solution Architect and on my QA team to guide and interpret this kind of information. Now that I have some hands-on experience with these tools, I feel equipped to review and understand the outputs of testing processes, participate in conversations with the dev team and even explain the work to our business stakeholders who might be seeking more information. I would also be able to have a conversation with the team about "dead-by-design" or cleaning up code so that it doesn't retain obsolete bits that might cause problems in the future.

- **Coverage metrics can create false confidence.** 91% line coverage looked reassuring on its own, but it told you nothing about whether the tests actually verified correctness. As a PM, this is a useful lesson for reading status reports: "high coverage" in a dashboard isn't the same claim as "well-tested," and knowing to ask which kind of coverage is a real skill.
- **Survived vs. no-coverage mutants demand different fixes.** Missing assertions on existing tests vs. entirely untested branches like the first-purchase discount are different issues and treating them the same wastes effort.
- **The dead-code finding reframes what mutation testing is for.** Lines 42-43 weren't a testing gap at all; they were unreachable defensive code the tool exposed. This means

that mutation testing isn't just a QA metric, it can highlight design and tech-debt issues a coverage report alone never would.

JUnit tests are only part of the story – even a broken clock is right twice a day. Strong tests should test the outputs not just whether the code runs. Running PITest will help detect faults in the code itself and help identify areas for improvement in ways that can be prioritized by the team based on risk, impact, and other dependencies.

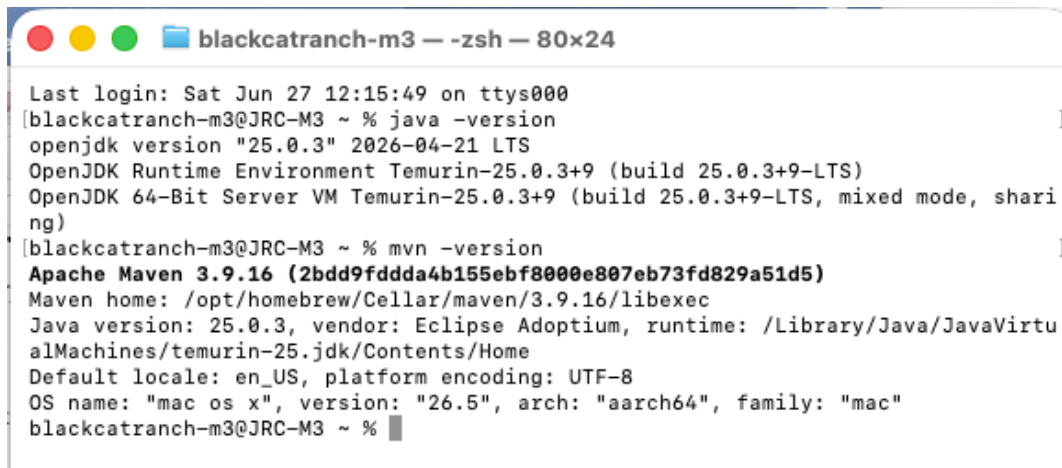
As a project/program manager – we face two big challenges:

1. **Resourcing for QA** always seems to be on the thin side as it is seen as something that happens “at the end” even in Agile environments.
2. **Allocation Conflicts:** Once we land a good QA tester, they face more demands for their time in CICD environments.
3. **Retention of QA professionals:** In recent years, it seems that my employers hire “freshers” right out of college who are seeking to move up into more senior roles quickly leaving gaps in knowledge as well as incentive to resolve tech-debt-related issues.

Being able to interpret the types of issues raised with mutation testing helps better prioritize where we allocate the QA team time, and may also offer opportunities for cross-training other team members (ie, front end dev with bench time may be able to help resolve certain types of issues).

APPENDIX 1: Technical Implementation

Figure 1. Java & Maven Verification in Terminal



```
blackcatranch-m3 — -zsh — 80x24

Last login: Sat Jun 27 12:15:49 on ttys000
blackcatranch-m3@JRC-M3 ~ % java -version
openjdk version "25.0.3" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-25.0.3+9 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.3+9 (build 25.0.3+9-LTS, mixed mode, sharing)
blackcatranch-m3@JRC-M3 ~ % mvn -version
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: /opt/homebrew/Cellar/maven/3.9.16/libexec
Java version: 25.0.3, vendor: Eclipse Adoptium, runtime: /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home
Default locale: en_US, platform encoding: UTF-8
OS name: "mac os x", version: "26.5", arch: "aarch64", family: "mac"
blackcatranch-m3@JRC-M3 ~ %
```

Figure 2. Project File Structure Verification in Finder

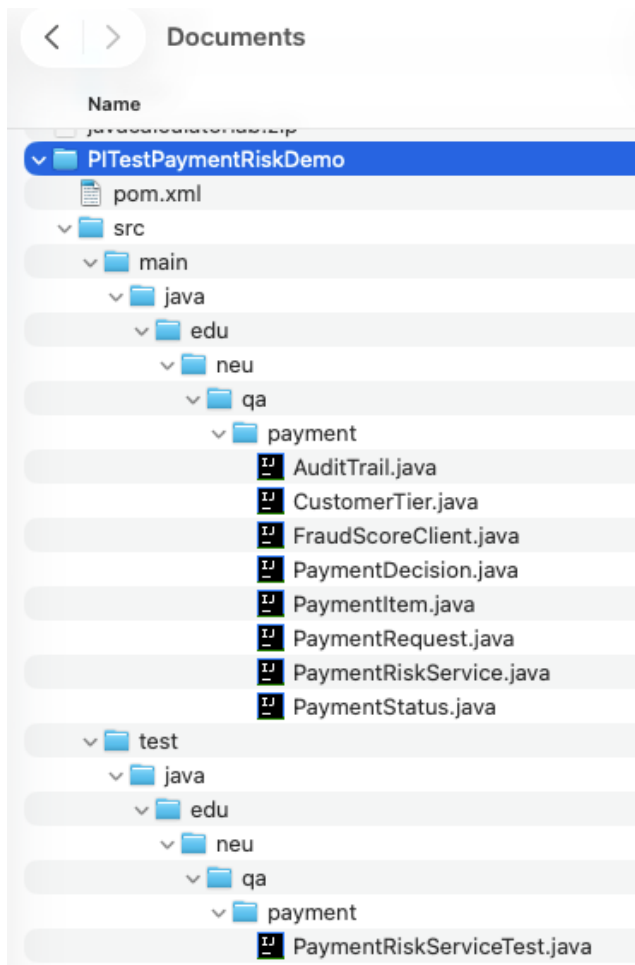
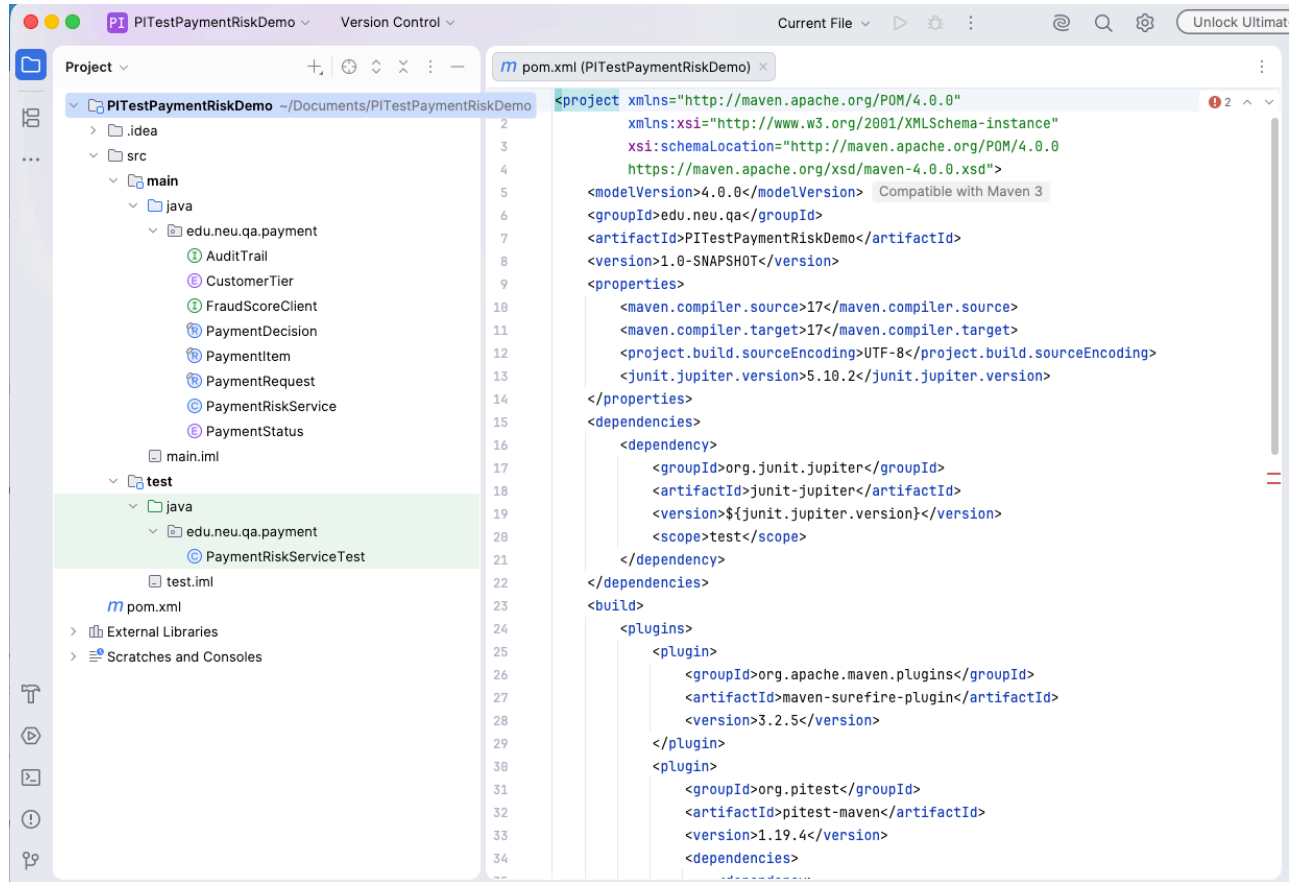


Figure 3. Project Imported into IntelliJ IDEA



APPENDIX 2: Testing – JUnit & PITest – Round 1

Figure 4. Results from “Run the Normal Junit Tests”

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.3/junit-platform-engine-1.9.3.jar
at 2.2 MB/s)
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running edu.neu.qa.payment.PaymentRiskServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in edu.neu.qa.payment.PaymentRiskServiceTest
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 3.151 s
[INFO] Finished at: 2026-07-09T18:27:27-07:00
[INFO] -----

blackcatranch-m3@JRC-M3 PITestPaymentRiskDemo %
```

Figure 5. Results from “Run PITest Mutation Testing”

```
=====
- Statistics
=====
>> Line Coverage (for mutated classes only): 51/56 (91%)
>> 9 tests examined
>> Generated 106 mutations Killed 39 (37%)
>> Mutations with no coverage 6. Test strength 39%
>> Ran 263 tests (2.48 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.461 s
[INFO] Finished at: 2026-07-09T18:29:10-07:00
[INFO] -----

blackcatranch-m3@JRC-M3 PITestPaymentRiskDemo %
```

Figure 6. Pit Test Coverage Report Dashboard

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	91% 51/56	37% 39/106	39% 39/100

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
edu.neu.qa.payment	1	91% 51/56	37% 39/106	39% 39/100

Figure 7. Full Report for PaymentRiskService.java – Round 1
PaymentRiskService.java

```
1 package edu.neu.qa.payment;
2
3 import java.util.List;
4 import java.util.Objects;
5
6 import static edu.neu.qa.payment.PaymentStatus.APPROVED;
7 import static edu.neu.qa.payment.PaymentStatus.MANUAL_REVIEW;
8 import static edu.neu.qa.payment.PaymentStatus.REJECTED;
9
10 public class PaymentRiskService {
11
12     private static final double HIGH_RISK_THRESHOLD = 0.85;
13     private static final double MANUAL_REVIEW_AMOUNT_LIMIT = 2500.00;
14     private static final double FREE_SHIPPING_THRESHOLD = 100.00;
15     private static final double STANDARD_SHIPPING_FEE = 7.99;
16     private static final int BULK_QUANTITY = 10;
17     private static final double GOLD_DISCOUNT_RATE = 0.05;
18     private static final double BULK_DISCOUNT_RATE = 0.10;
19     private static final double FIRST_PURCHASE_DISCOUNT = 12.00;
20     private static final double MAX_DISCOUNT_RATE = 0.25;
21
22     private final FraudScoreClient fraudScoreClient;
23     private final AuditTrail auditTrail;
24
25     public PaymentRiskService(FraudScoreClient fraudScoreClient, AuditTrail auditTrail) {
26         1 this.fraudScoreClient = Objects.requireNonNull(fraudScoreClient);
27         1 this.auditTrail = Objects.requireNonNull(auditTrail);
28     }
29
30     public PaymentDecision authorize(PaymentRequest request) {
31         4 if (request == null || request.items().isEmpty()) {
32             1 auditTrail.record("REJECTED_EMPTY_PAYMENT");
33             3 return new PaymentDecision(REJECTED, 0.00, 0, "Payment request has no payable items");
34         }
35
36         2 double subtotal = calculateSubtotal(request.items());
37         1 double discount = calculateDiscount(request, subtotal);
38         1 double shipping = calculateShipping(subtotal);
39         3 double authorizedAmount = round(subtotal - discount + shipping);
```

Figure 8. Full Report for PaymentRiskService.java – Round 1

```
40
41 3 if (authorizedAmount <= 0) {
42 2 auditTrail.record("REJECTED INVALID AMOUNT:" + request.customerId());
43 2 return new PaymentDecision(REJECTED, authorizedAmount, 0, "Authorized amount is invalid");
44 }
45
46 1 double fraudScore = fraudScoreClient.score(
47 1 request.customerId(),
48 authorizedAmount,
49 1 request.country()
50 );
51
52 2 int rewardPoints = calculateRewardPoints(request.items(), authorizedAmount);
53
54 6 if (fraudScore >= HIGH_RISK_THRESHOLD || authorizedAmount > MANUAL_REVIEW_AMOUNT_LIMIT) {
55 2 auditTrail.record("MANUAL REVIEW:" + request.customerId());
56 1 return new PaymentDecision(
57 MANUAL_REVIEW,
58 authorizedAmount,
59 rewardPoints,
60 "Payment requires manual review"
61 );
62 }
63
64 2 auditTrail.record("APPROVED:" + request.customerId());
65 1 return new PaymentDecision(
66 APPROVED,
67 authorizedAmount,
68 rewardPoints,
69 "Payment approved"
70 );
71 }
72
73 double calculateSubtotal(List<PaymentItem> items) {
74 1 double total = 0.00;
75 for (PaymentItem item : items) {
76 1 if (item == null) {
77 throw new IllegalArgumentException("Payment item cannot be null");
78 }
79 3 if (item.quantity() <= 0) {
80 throw new IllegalArgumentException("Quantity must be greater than zero");
81 }
82 4 if (item.unitPrice() < 0) {
83 throw new IllegalArgumentException("Unit price cannot be negative");
```

Figure 9. Full Report for PaymentRiskService.java – Round 1

```
84     }
85     4 total += item.quantity() * item.unitPrice();
86     }
87     2 return round(total);
88 }
89
90 double calculateDiscount(PaymentRequest request, double subtotal) {
91     1 double discount = 0.00;
92
93     2 if (request.tier() == CustomerTier.GOLD) {
94         3 discount += subtotal * GOLD_DISCOUNT_RATE;
95     }
96
97     1 boolean hasBulkItem = false;
98     1 for (PaymentItem item : request.items()) {
99         4 if (item.quantity() >= BULK_QUANTITY) {
100         1 hasBulkItem = true;
101     }
102 }
103     1 if (hasBulkItem) {
104         3 discount += subtotal * BULK_DISCOUNT_RATE;
105     }
106
107     2 if (request.firstPurchase()) {
108         2 discount += FIRST_PURCHASE_DISCOUNT;
109     }
110
111     2 double maximumAllowedDiscount = subtotal * MAX_DISCOUNT_RATE;
112     3 return round(Math.min(discount, maximumAllowedDiscount));
113 }
114
115 double calculateShipping(double subtotal) {
116     3 if (subtotal >= FREE_SHIPPING_THRESHOLD) {
117         1 return 0.00;
118     }
119     2 return STANDARD_SHIPPING_FEE;
120 }
121
122 int calculateRewardPoints(List<PaymentItem> items, double authorizedAmount) {
123     1 int itemCount = 0;
124     4 for (int i = 0; i < items.size(); i++) {
125         3 itemCount += items.get(i).quantity();
126     }
127     4 return itemCount + (int) (authorizedAmount / 20);
}
```

Figure 10. Full Report for PaymentRiskService.java – Round 1

128	}
129	
130	double round(double value) {
131	return Math.round(value * 100.0) / 100.0;
132	}
133	}
Mutations	
26	1. removed call to java/util/Objects::requireNonNull → KILLED
27	1. removed call to java/util/Objects::requireNonNull → KILLED
	1. negated conditional → KILLED
	2. negated conditional → KILLED
31	3. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
	4. removed call to java/util/List::isEmpty → KILLED
32	1. removed call to edu/neu/qa/payment/AuditTrail::record → SURVIVED Covering tests
	1. Substituted 0 with 1 → SURVIVED Covering tests
33	2. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
	3. Substituted 0.0 with 1.0 → SURVIVED Covering tests
36	1. removed call to edu/neu/qa/payment/PaymentRiskService::calculateSubtotal → KILLED
	2. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
37	1. removed call to edu/neu/qa/payment/PaymentRiskService::calculateDiscount → SURVIVED Covering tests
38	1. removed call to edu/neu/qa/payment/PaymentRiskService::calculateShipping → SURVIVED Covering tests
	1. Replaced double addition with subtraction → SURVIVED Covering tests
39	2. Replaced double subtraction with addition → SURVIVED Covering tests
	3. removed call to edu/neu/qa/payment/PaymentRiskService::round → KILLED
	1. negated conditional → KILLED
41	2. changed conditional boundary → SURVIVED Covering tests
	3. Substituted 0.0 with 1.0 → SURVIVED Covering tests
42	1. removed call to edu/neu/qa/payment/AuditTrail::record → NO_COVERAGE
	2. removed call to edu/neu/qa/payment/PaymentRequest::customerId → NO_COVERAGE
43	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → NO_COVERAGE
	2. Substituted 0 with 1 → NO_COVERAGE
46	1. removed call to edu/neu/qa/payment/FraudScoreClient::score → KILLED
47	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → SURVIVED Covering tests
49	1. removed call to edu/neu/qa/payment/PaymentRequest::country → SURVIVED Covering tests
52	1. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
	2. removed call to edu/neu/qa/payment/PaymentRiskService::calculateRewardPoints → SURVIVED Covering tests
	1. changed conditional boundary → SURVIVED Covering tests
	2. negated conditional → KILLED
54	3. Substituted 2500.0 with 1.0 → KILLED
	4. negated conditional → KILLED
	5. changed conditional boundary → SURVIVED Covering tests
	6. Substituted 0.85 with 1.0 → KILLED
55	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → SURVIVED Covering tests
	2. removed call to edu/neu/qa/payment/AuditTrail::record → SURVIVED Covering tests
56	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED

Figure 11. Full Report for PaymentRiskService.java – Round 1

56	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
64	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → SURVIVED Covering tests 2. removed call to edu/neu/qa/payment/AuditTrail::record → SURVIVED Covering tests
65	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
74	1. Substituted 0.0 with 1.0 → SURVIVED Covering tests
76	1. negated conditional → KILLED
	1. changed conditional boundary → KILLED
79	2. negated conditional → KILLED 3. removed call to edu/neu/qa/payment/PaymentItem::quantity → KILLED
	1. Substituted 0.0 with 1.0 → SURVIVED Covering tests 2. removed call to edu/neu/qa/payment/PaymentItem::unitPrice → SURVIVED Covering tests 3. changed conditional boundary → SURVIVED Covering tests 4. negated conditional → KILLED
82	1. removed call to edu/neu/qa/payment/PaymentItem::unitPrice → KILLED 2. Replaced double multiplication with division → KILLED 3. Replaced double addition with subtraction → KILLED 4. removed call to edu/neu/qa/payment/PaymentItem::quantity → KILLED
85	1. removed call to edu/neu/qa/payment/PaymentRiskService::round → KILLED 2. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::calculateSubtotal → KILLED
87	1. Substituted 0.0 with 1.0 → SURVIVED Covering tests
91	1. negated conditional → SURVIVED Covering tests
93	2. removed call to edu/neu/qa/payment/PaymentRequest::tier → SURVIVED Covering tests
	1. Substituted 0.05 with 1.0 → SURVIVED Covering tests 2. Replaced double multiplication with division → SURVIVED Covering tests 3. Replaced double addition with subtraction → SURVIVED Covering tests
94	1. Substituted 0 with 1 → SURVIVED Covering tests
97	1. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
98	1. negated conditional → SURVIVED Covering tests 2. Substituted 10 with 11 → SURVIVED Covering tests 3. changed conditional boundary → SURVIVED Covering tests 4. removed call to edu/neu/qa/payment/PaymentItem::quantity → SURVIVED Covering tests
99	1. Substituted 1 with 0 → SURVIVED Covering tests
100	1. negated conditional → SURVIVED Covering tests
103	1. Replaced double multiplication with division → SURVIVED Covering tests 2. Replaced double addition with subtraction → SURVIVED Covering tests 3. Substituted 0.1 with 1.0 → SURVIVED Covering tests
104	1. removed call to edu/neu/qa/payment/PaymentRequest::firstPurchase → SURVIVED Covering tests 2. negated conditional → SURVIVED Covering tests
107	1. Substituted 12.0 with 1.0 → NO_COVERAGE 2. Replaced double addition with subtraction → NO_COVERAGE
108	1. Substituted 0.25 with 1.0 → SURVIVED Covering tests 2. Replaced double multiplication with division → SURVIVED Covering tests
111	1. removed call to java/lang/Math::min → SURVIVED Covering tests 2. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::calculateDiscount → SURVIVED Covering tests 3. removed call to edu/neu/qa/payment/PaymentRiskService::round → SURVIVED Covering tests
112	1. Substituted 100.0 with 1.0 → SURVIVED Covering tests 2. changed conditional boundary → SURVIVED Covering tests 3. negated conditional → SURVIVED Covering tests
116	1. Substituted 0.0 with 1.0 → SURVIVED Covering tests
117	

Figure 12. Full Report for PaymentRiskService.java – Round 1

117	1. Substituted 0.0 with 1.0 → SURVIVED Covering tests
119	1. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::calculateShipping → SURVIVED Covering tests 2. Substituted 7.99 with 1.0 → SURVIVED Covering tests
123	1. Substituted 0 with 1 → SURVIVED Covering tests 1. removed call to java/util/List::size → SURVIVED Covering tests
124	2. changed conditional boundary → KILLED 3. negated conditional → SURVIVED Covering tests 4. Substituted 0 with 1 → SURVIVED Covering tests
125	1. removed call to edu/neu/qa/payment/PaymentItem::quantity → SURVIVED Covering tests 2. Replaced integer addition with subtraction → SURVIVED Covering tests 3. removed call to java/util/List::get → KILLED
127	1. Replaced integer addition with subtraction → SURVIVED Covering tests 2. replaced int return with 0 for edu/neu/qa/payment/PaymentRiskService::calculateRewardPoints → SURVIVED Covering tests 3. Replaced double division with multiplication → SURVIVED Covering tests 4. Substituted 20.0 with 1.0 → SURVIVED Covering tests
131	1. Replaced double division with multiplication → KILLED 2. Substituted 100.0 with 1.0 → KILLED 3. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::round → KILLED 4. Replaced double multiplication with division → KILLED 5. removed call to java/lang/Math::round → KILLED 6. Substituted 100.0 with 1.0 → KILLED

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INLINE_CONSTS
- MATH
- NEGATE_CONDITIONALS
- NON_VOID_METHOD_CALLS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:sendsHighFraudScorePaymentToManualReview()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:sendsLargePaymentToManualReview()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:approvesSmallStandardPaymentWithShipping()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:approvesLowRiskGoldBulkPayment()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:rejectsEmptyPaymentRequest()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:rejectsZeroQuantityItem()] (5 ms)

Figure 13. Full Report for PaymentRiskService.java – Round 1

PaymentRiskService.java

[illegible]

Active mutators

- CONDITIONALS, BOUNDARIES
- EMPTY RETURNS
- FALSE RETURNS
- INCREMENTS
- INLINE_CONSTS
- MATH
- NEGATE CONDITIONALS
- NON_VOID_METHOD_CALLS
- NULL RETURNS
- PRIMITIVE RETURNS
- TRUE RETURNS
- VOID METHOD CALLS

Tests examined

•	rela acea cu payment.PaymentRiskServiceTest(jugine, jani papir)	class rela acea cu payment.PaymentRiskServiceTest	method sendBrightSignalToPaymentToMoodReview() (1 ms)
•	rela acea cu payment.PaymentRiskServiceTest(jugine, jani papir)	class rela acea cu payment.PaymentRiskServiceTest	method sendBrightSignalToPaymentToMoodReview() (1 ms)
•	rela acea cu payment.PaymentRiskServiceTest(jugine, jani papir)	class rela acea cu payment.PaymentRiskServiceTest	method approveAndSendToPaymentReviewWithShipping() (1 ms)
•	rela acea cu payment.PaymentRiskServiceTest(jugine, jani papir)	class rela acea cu payment.PaymentRiskServiceTest	method approveAndSendToPaymentReviewWithShipping() (1 ms)
•	rela acea cu payment.PaymentRiskServiceTest(jugine, jani papir)	class rela acea cu payment.PaymentRiskServiceTest	method approveAndSendToPaymentReviewWithShipping() (1 ms)

Exported by [PDF](#) 1.19.6

APPENDIX 3: Summary of Test Changes

1. Strengthened assertions on existing tests (6 tests)

The original six tests only asserted `decision.status()`. This let mutants survive because the test would still pass even if the *wrong number* came back, as long as the status enum matched. Each test now also asserts:

Test	Added assertions	Mutants killed
<code>approvesLowRiskGoldBulkPayment</code>	<code>exact authorizedAmount (425.00), exact rewardPoints (31), audit log contains "APPROVED:CUST-100"</code>	Line 39 (+/- swap), lines 94/104 (discount rate math), line 52/127 (reward points), lines 64-65 (audit call)
<code>sendsHighFraudScorePaymentToManualReview</code>	<code>exact amount (240.00), exact reward points (14), exact reason string, audit contains "MANUAL_REVIEW:CUST-200"</code>	Lines 55-56 (audit + return value), line 54 (boundary constants)
<code>sendsLargePaymentToManualReview</code>	<code>exact amount (3000.00), exact reward points (153), audit contains "MANUAL_REVIEW:CUST-300"</code>	Line 54 (the amount-based OR branch specifically, isolated from fraud score since score is low here)
<code>rejectsEmptyPaymentRequest</code>	<code>exact amount (0.00), exact reward points (0), exact reason string, audit contains "REJECTED_EMPTY_PAYMENT", and asserts the fraud client was never called (callCount == 0)</code>	Lines 32-33 (audit call + return value substitutions)
<code>rejectsZeroQuantityItem</code>	<code>exact exception message</code>	Line 79 (boundary/negation on quantity check)
<code>approvesSmallStandardPaymentWithShipping</code>	<code>exact amount (47.99), exact reward points (3), audit contains "APPROVED:CUST-600"</code>	Lines 116-119 (shipping fee constant/branch)

Report framing: "Existing tests verified the correct *category* of outcome but not the correct *value*, allowing arithmetic and audit-logging mutants to survive undetected."

2. New tests for a completely uncovered branch

firstPurchaseDiscountIsApplied — Line 108 (`discount += FIRST_PURCHASE_DISCOUNT`) had NO_COVERAGE: no test ever set `firstPurchase = true`. This test does, and asserts the discount calculation directly (`calculateDiscount(request, 50.00) == 12.00`) as well as the downstream effect on the final authorized amount.

3. New tests isolating discount logic (package-private helper calls)

Rather than relying on `authorize()` end-to-end (which mixes subtotal, discount, and shipping together, letting some mutants hide behind coincidental cancellation), these call `calculateDiscount()` directly:

- **discountIsCappedAtMaximumRate** — combines gold + bulk + first-purchase discounts so the raw total (27.00) exceeds the 25% cap (25.00 on a 100.00 subtotal), and asserts the capped value exactly. Targets line 111-112 (`Math.min` cap logic).
- **goldTierDiscountAppliedDirectly** — same items/subtotal, only the tier differs (STANDARD vs GOLD), isolating the gold-tier conditional (line 93) from bulk/first-purchase logic.
- **bulkDiscountBoundary** — quantity 9 vs. quantity 10 on the same unit price, isolating the `>= BULK_QUANTITY` boundary (line 99) from other discount types.

4. New boundary-value tests

Mutation testing showed several "changed conditional boundary" mutants surviving — meaning the tests never exercised the exact threshold value where behavior changes:

- **shippingBoundary** — `calculateShipping(99.99)` vs `calculateShipping(100.00)`, targeting line 116's `>=` comparison.
- **fraudScoreBoundaryTriggersManualReviewAtExactThreshold** — fraud score exactly 0.85 (should trigger review) vs. 0.84 (should not), with the transaction amount held low so this isolates the fraud-score half of the OR condition on line 54 from the amount half.
- **amountBoundaryAtManualReviewLimit** — authorized amount exactly 2500.00 (should still be approved, since the check is `>`, not `>=`) vs. 2500.01 (should trigger review), isolating the amount half of that same OR condition.

5. New tests for input-validation edges

- **subtotalAllowsZeroPriceButRejectsNegativePrice** — confirms `unitPrice = 0.00` is accepted (no exception) and `unitPrice = -0.01` throws with the exact expected message, targeting line 82's boundary and negation mutants.

6. New test for internal math directly

- **rewardPointsFlooringAndMultiItemSum** — three items with distinct quantities (2, 3, 4) confirm the loop sums correctly across multiple elements (targeting line 124-125's loop/index mutants), and two amount values straddling a /20 boundary (39.99 vs 40.00) confirm integer-division truncation behaves correctly rather than rounding (targeting line 127).

7. New test verifying mock interaction arguments

- **fraudClientReceivesCorrectCustomerIdAmountAndCountry** — previously the stub returned a fixed fraud score regardless of what was passed in, so mutants removing the `customerId()/country()` argument calls (lines 47, 49) survived. This test captures and asserts the exact arguments the service passed to `fraudScoreClient.score(...)`.

8. Known unaddressed gap (for the report's limitations section)

Lines 42-43 (`authorizedAmount <= 0 → REJECTED`) remain uncovered. Analysis of the surrounding validation logic shows this branch is **unreachable through the public API**: `quantity > 0` and `unitPrice >= 0` guarantee `subtotal >= 0`; the discount is capped at 25% of subtotal, so `subtotal - discount >= 0`; and shipping is always `>= 0` (specifically 7.99 in the one scenario where subtotal is small). No combination of valid inputs can drive `authorizedAmount` below zero. This is worth documenting as **defensive/dead code** rather than a testing gap — closing it would require either changing validation rules or restructuring the method to allow dependency injection of intermediate values.

SOURCE: <https://claude.ai/share/0ff11bbc-8926-404e-8df6-593a2a7ce357>

APPENDIX 4: Testing - Pit Test - Round 2

Figure 14. Dashboard for PaymentRiskService.java – Round 2

Pit Test Coverage Report

Package Summary

edu.neu.qa.payment

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	95% 53/56	94% 100/106	98% 100/102

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
PaymentRiskService.java	95% 53/56	94% 100/106	98% 100/102

Report generated by [PIT](#) 1.19.4

Figure 15. Full Report for PaymentRiskService.java – Round 2
PaymentRiskService.java

```
1 package edu.neu.qa.payment;
2
3 import java.util.List;
4 import java.util.Objects;
5
6 import static edu.neu.qa.payment.PaymentStatus.APPROVED;
7 import static edu.neu.qa.payment.PaymentStatus.MANUAL_REVIEW;
8 import static edu.neu.qa.payment.PaymentStatus.REJECTED;
9
10 public class PaymentRiskService {
11
12     private static final double HIGH_RISK_THRESHOLD = 0.85;
13     private static final double MANUAL_REVIEW_AMOUNT_LIMIT = 2500.00;
14     private static final double FREE_SHIPPING_THRESHOLD = 100.00;
15     private static final double STANDARD_SHIPPING_FEE = 7.99;
16     private static final int BULK_QUANTITY = 10;
17     private static final double GOLD_DISCOUNT_RATE = 0.05;
18     private static final double BULK_DISCOUNT_RATE = 0.10;
19     private static final double FIRST_PURCHASE_DISCOUNT = 12.00;
20     private static final double MAX_DISCOUNT_RATE = 0.25;
21
22     private final FraudScoreClient fraudScoreClient;
23     private final AuditTrail auditTrail;
24
25     public PaymentRiskService(FraudScoreClient fraudScoreClient, AuditTrail auditTrail) {
26         1 this.fraudScoreClient = Objects.requireNonNull(fraudScoreClient);
27         1 this.auditTrail = Objects.requireNonNull(auditTrail);
28     }
29
30     public PaymentDecision authorize(PaymentRequest request) {
31         4 if (request == null || request.items().isEmpty()) {
32             1 auditTrail.record("REJECTED_EMPTY_PAYMENT");
33             3 return new PaymentDecision(REJECTED, 0.00, 0, "Payment request has no payable items");
34         }
35
36         2 double subtotal = calculateSubtotal(request.items());
37         1 double discount = calculateDiscount(request, subtotal);
38         1 double shipping = calculateShipping(subtotal);
39         3 double authorizedAmount = round(subtotal - discount + shipping);
40     }
```

Figure 16. Full Report for PaymentRiskService.java – Round 2

```
41 3 if (authorizedAmount <= 0) {
42 2 auditTrail.record("REJECTED_INVALID_AMOUNT:" + request.customerId());
43 2 return new PaymentDecision(REJECTED, authorizedAmount, 0, "Authorized amount is invalid");
44 }
45
46 1 double fraudScore = fraudScoreClient.score(
47 1 request.customerId(),
48 authorizedAmount,
49 1 request.country()
50 );
51
52 2 int rewardPoints = calculateRewardPoints(request.items(), authorizedAmount);
53
54 6 if (fraudScore >= HIGH_RISK_THRESHOLD || authorizedAmount > MANUAL_REVIEW_AMOUNT_LIMIT) {
55 2 auditTrail.record("MANUAL_REVIEW:" + request.customerId());
56 1 return new PaymentDecision(
57 MANUAL_REVIEW,
58 authorizedAmount,
59 rewardPoints,
60 "Payment requires manual review"
61 );
62 }
63
64 2 auditTrail.record("APPROVED:" + request.customerId());
65 1 return new PaymentDecision(
66 APPROVED,
67 authorizedAmount,
68 rewardPoints,
69 "Payment approved"
70 );
71 }
72
73 double calculateSubtotal(List<PaymentItem> items) {
74 1 double total = 0.00;
75 for (PaymentItem item : items) {
76 1 if (item == null) {
77 throw new IllegalArgumentException("Payment item cannot be null");
78 }
79 3 if (item.quantity() <= 0) {
80 throw new IllegalArgumentException("Quantity must be greater than zero");
81 }
82 4 if (item.unitPrice() < 0) {
83 throw new IllegalArgumentException("Unit price cannot be negative");
84 }
```

Figure 17. Full Report for PaymentRiskService.java – Round 2

```
85 4      total += item.quantity() * item.unitPrice();
86      }
87 2      return round(total);
88  }
89
90  double calculateDiscount(PaymentRequest request, double subtotal) {
91  1      double discount = 0.00;
92
93  2      if (request.tier() == CustomerTier.GOLD) {
94  3          discount += subtotal * GOLD_DISCOUNT_RATE;
95  }
96
97  1      boolean hasBulkItem = false;
98  1      for (PaymentItem item : request.items()) {
99  4          if (item.quantity() >= BULK_QUANTITY) {
100  1              hasBulkItem = true;
101  }
102  }
103  1      if (hasBulkItem) {
104  3          discount += subtotal * BULK_DISCOUNT_RATE;
105  }
106
107  2      if (request.firstPurchase()) {
108  2          discount += FIRST_PURCHASE_DISCOUNT;
109  }
110
111  2      double maximumAllowedDiscount = subtotal * MAX_DISCOUNT_RATE;
112  3      return round(Math.min(discount, maximumAllowedDiscount));
113  }
114
115  double calculateShipping(double subtotal) {
116  3      if (subtotal >= FREE_SHIPPING_THRESHOLD) {
117  1          return 0.00;
118  }
119  2      return STANDARD_SHIPPING_FEE;
120  }
121
122  int calculateRewardPoints(List<PaymentItem> items, double authorizedAmount) {
123  1      int itemCount = 0;
124  4      for (int i = 0; i < items.size(); i++) {
125  3          itemCount += items.get(i).quantity();
126  }
127  4      return itemCount + (int) (authorizedAmount / 20);
128  }
```

Figure 18. Full Report for PaymentRiskService.java – Round 2

129	
130	double round(double value) {
131	return Math.round(value * 100.0) / 100.0;
132	}
133	}
Mutations	
26	1. removed call to java/util/Objects::requireNonNull → KILLED
27	1. removed call to java/util/Objects::requireNonNull → KILLED
	1. negated conditional → KILLED
	2. negated conditional → KILLED
31	3. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
	4. removed call to java/util/List::isEmpty → KILLED
32	1. removed call to edu/neu/qa/payment/AuditTrail::record → KILLED
	1. Substituted 0 with 1 → KILLED
33	2. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
	3. Substituted 0.0 with 1.0 → KILLED
36	1. removed call to edu/neu/qa/payment/PaymentRiskService::calculateSubtotal → KILLED
	2. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
37	1. removed call to edu/neu/qa/payment/PaymentRiskService::calculateDiscount → KILLED
38	1. removed call to edu/neu/qa/payment/PaymentRiskService::calculateShipping → KILLED
	1. Replaced double addition with subtraction → KILLED
39	2. Replaced double subtraction with addition → KILLED
	3. removed call to edu/neu/qa/payment/PaymentRiskService::round → KILLED
	1. negated conditional → KILLED
41	2. changed conditional boundary → SURVIVED Covering tests
	3. Substituted 0.0 with 1.0 → SURVIVED Covering tests
42	1. removed call to edu/neu/qa/payment/AuditTrail::record → NO COVERAGE
	2. removed call to edu/neu/qa/payment/PaymentRequest::customerId → NO COVERAGE
43	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → NO_COVERAGE
	2. Substituted 0 with 1 → NO_COVERAGE
46	1. removed call to edu/neu/qa/payment/FraudScoreClient::score → KILLED
47	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → KILLED
49	1. removed call to edu/neu/qa/payment/PaymentRequest::country → KILLED
	1. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
52	2. removed call to edu/neu/qa/payment/PaymentRiskService::calculateRewardPoints → KILLED
	1. changed conditional boundary → KILLED
	2. negated conditional → KILLED
	3. Substituted 2500.0 with 1.0 → KILLED
54	4. negated conditional → KILLED
	5. changed conditional boundary → KILLED
	6. Substituted 0.85 with 1.0 → KILLED
55	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → KILLED
	2. removed call to edu/neu/qa/payment/AuditTrail::record → KILLED
56	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
57	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → KILLED

Figure 19. Full Report for PaymentRiskService.java – Round 2

56	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
64	1. removed call to edu/neu/qa/payment/PaymentRequest::customerId → KILLED
	2. removed call to edu/neu/qa/payment/AuditTrail::record → KILLED
65	1. replaced return value with null for edu/neu/qa/payment/PaymentRiskService::authorize → KILLED
74	1. Substituted 0.0 with 1.0 → KILLED
76	1. negated conditional → KILLED
	1. changed conditional boundary → KILLED
79	2. negated conditional → KILLED
	3. removed call to edu/neu/qa/payment/PaymentItem::quantity → KILLED
	1. Substituted 0.0 with 1.0 → KILLED
82	2. removed call to edu/neu/qa/payment/PaymentItem::unitPrice → KILLED
	3. changed conditional boundary → KILLED
	4. negated conditional → KILLED
	1. removed call to edu/neu/qa/payment/PaymentItem::unitPrice → KILLED
85	2. Replaced double multiplication with division → KILLED
	3. Replaced double addition with subtraction → KILLED
	4. removed call to edu/neu/qa/payment/PaymentItem::quantity → KILLED
	1. removed call to edu/neu/qa/payment/PaymentRiskService::round → KILLED
87	2. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::calculateSubtotal → KILLED
	1. Substituted 0.0 with 1.0 → KILLED
93	1. negated conditional → KILLED
	2. removed call to edu/neu/qa/payment/PaymentRequest::tier → KILLED
	1. Substituted 0.05 with 1.0 → KILLED
94	2. Replaced double multiplication with division → KILLED
	3. Replaced double addition with subtraction → KILLED
97	1. Substituted 0 with 1 → KILLED
98	1. removed call to edu/neu/qa/payment/PaymentRequest::items → KILLED
	1. negated conditional → KILLED
	2. Substituted 10 with 11 → KILLED
99	3. changed conditional boundary → KILLED
	4. removed call to edu/neu/qa/payment/PaymentItem::quantity → KILLED
100	1. Substituted 1 with 0 → KILLED
103	1. negated conditional → KILLED
	1. Replaced double multiplication with division → KILLED
104	2. Replaced double addition with subtraction → KILLED
	3. Substituted 0.1 with 1.0 → KILLED
107	1. removed call to edu/neu/qa/payment/PaymentRequest::firstPurchase → KILLED
	2. negated conditional → KILLED
108	1. Substituted 12.0 with 1.0 → KILLED
	2. Replaced double addition with subtraction → KILLED
111	1. Substituted 0.25 with 1.0 → KILLED
	2. Replaced double multiplication with division → KILLED
112	1. removed call to java/lang/Math::min → KILLED
	2. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::calculateDiscount → KILLED
	3. removed call to edu/neu/qa/payment/PaymentRiskService::round → KILLED
	1. Substituted 100.0 with 1.0 → KILLED
116	2. changed conditional boundary → KILLED
	3. negated conditional → KILLED

Figure 20. Full Report for PaymentRiskService.java – Round 2

117	1. Substituted 0.0 with 1.0 → KILLED
119	1. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::calculateShipping → KILLED
	2. Substituted 7.99 with 1.0 → KILLED
123	1. Substituted 0 with 1 → KILLED
	1. removed call to java/util/List::size → KILLED
124	2. changed conditional boundary → KILLED
	3. negated conditional → KILLED
	4. Substituted 0 with 1 → KILLED
125	1. removed call to edu/neu/qa/payment/PaymentItem::quantity → KILLED
	2. Replaced integer addition with subtraction → KILLED
	3. removed call to java/util/List::get → KILLED
127	1. Replaced integer addition with subtraction → KILLED
	2. replaced int return with 0 for edu/neu/qa/payment/PaymentRiskService::calculateRewardPoints → KILLED
	3. Replaced double division with multiplication → KILLED
	4. Substituted 20.0 with 1.0 → KILLED
	1. Replaced double division with multiplication → KILLED
	2. Substituted 100.0 with 1.0 → KILLED
131	3. replaced double return with 0.0d for edu/neu/qa/payment/PaymentRiskService::round → KILLED
	4. Replaced double multiplication with division → KILLED
	5. removed call to java/lang/Math::round → KILLED
	6. Substituted 100.0 with 1.0 → KILLED

Figure 21. Full Report for PaymentRiskService.java – Round 2

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INLINE_CONSTS
- MATH
- NEGATE_CONDITIONALS
- NON_VOID_METHOD_CALLS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Tests examined

- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:sendsLargePaymentToManualReview()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:fraudScoreBoundaryTriggersManualReviewAtExactThreshold()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:sendsHighFraudScorePaymentToManualReview()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:amountBoundaryAtManualReviewLimit()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:shippingBoundary()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:approvesSmallStandardPaymentWithShipping()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:fraudClientReceivesCorrectCustomerIdAmountAndCountry()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:firstPurchaseDiscountIsApplied()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:discountIsCappedAtMaximumRate()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:approvesLowRiskGoldBulkPayment()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:bulkDiscountBoundary()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:goldTierDiscountAppliedDirectly()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:subtotalAllowsZeroPriceButRejectsNegativePrice()] (5 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:rejectsZeroQuantityItem()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:rejectsEmptyPaymentRequest()] (0 ms)
- edu.neu.qa.payment.PaymentRiskServiceTest.[engine:junit-jupiter]/[class:edu.neu.qa.payment.PaymentRiskServiceTest]/[method:rewardPointsFlooringAndMultiItemSum()] (0 ms)

Figure 22. Full Report for PaymentRiskService.java – Round 2

PaymentRiskService.java

[illegible]

Mutations

[illegible]

Active mutators

- CONDITIONALS, BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVOKE_CONSTS
- MATH
- NEGATE_CONDITIONALS
- NON_VOID_METHOD_CALLS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

Taste examined

```

1  # test: username
2  # test: password
3  # test: email
4  # test: phone
5  # test: address
6  # test: city
7  # test: state
8  # test: zip
9  # test: country
10 # test: gender
11 # test: age
12 # test: height
13 # test: weight
14 # test: eye_color
15 # test: hair_color
16 # test: skin_color
17 # test: hair_style
18 # test: hair_color_hex
19 # test: hair_style_hex
20 # test: skin_color_hex
21 # test: eye_color_hex
22 # test: hair_color_hex
23 # test: hair_style_hex
24 # test: skin_color_hex
25 # test: eye_color_hex
26 # test: hair_color_hex
27 # test: hair_style_hex
28 # test: skin_color_hex
29 # test: eye_color_hex
30 # test: hair_color_hex
31 # test: hair_style_hex
32 # test: skin_color_hex
33 # test: eye_color_hex
34 # test: hair_color_hex
35 # test: hair_style_hex
36 # test: skin_color_hex
37 # test: eye_color_hex
38 # test: hair_color_hex
39 # test: hair_style_hex
40 # test: skin_color_hex
41 # test: eye_color_hex
42 # test: hair_color_hex
43 # test: hair_style_hex
44 # test: skin_color_hex
45 # test: eye_color_hex
46 # test: hair_color_hex
47 # test: hair_style_hex
48 # test: skin_color_hex
49 # test: eye_color_hex
50 # test: hair_color_hex
51 # test: hair_style_hex
52 # test: skin_color_hex
53 # test: eye_color_hex
54 # test: hair_color_hex
55 # test: hair_style_hex
56 # test: skin_color_hex
57 # test: eye_color_hex
58 # test: hair_color_hex
59 # test: hair_style_hex
60 # test: skin_color_hex
61 # test: eye_color_hex
62 # test: hair_color_hex
63 # test: hair_style_hex
64 # test: skin_color_hex
65 # test: eye_color_hex
66 # test: hair_color_hex
67 # test: hair_style_hex
68 # test: skin_color_hex
69 # test: eye_color_hex
70 # test: hair_color_hex
71 # test: hair_style_hex
72 # test: skin_color_hex
73 # test: eye_color_hex
74 # test: hair_color_hex
75 # test: hair_style_hex
76 # test: skin_color_hex
77 # test: eye_color_hex
78 # test: hair_color_hex
79 # test: hair_style_hex
80 # test: skin_color_hex
81 # test: eye_color_hex
82 # test: hair_color_hex
83 # test: hair_style_hex
84 # test: skin_color_hex
85 # test: eye_color_hex
86 # test: hair_color_hex
87 # test: hair_style_hex
88 # test: skin_color_hex
89 # test: eye_color_hex
90 # test: hair_color_hex
91 # test: hair_style_hex
92 # test: skin_color_hex
93 # test: eye_color_hex
94 # test: hair_color_hex
95 # test: hair_style_hex
96 # test: skin_color_hex
97 # test: eye_color_hex
98 # test: hair_color_hex
99 # test: hair_style_hex
100 # test: skin_color_hex

```

Methodology and Use of AI Tools

I relied on the coursework “Setup and Code” guide, Class Connect and other Canvas & Coursera materials for CSYE6255.

Claude was used to extract the assignment sample code from the PDF to create the file structure for the sample project. The analysis, observations, and conclusions reflect the author's own hands-on experience with the tool and tasks analyzed. All content has been reviewed for accuracy and represents the author's understanding of the results.

Reference to Claude LLM conversation on sample code extraction:

<https://claude.ai/share/336a0a95-6ac2-4bde-83da-bd917fdb6fd>

Reference to Claude LLM review of PITest patterns and recommendations:

<https://claude.ai/share/0ff11bbc-8926-404e-8df6-593a2a7ce357>

Reference to Claude LLM conversation on draft and rubric compliance:

<https://claude.ai/share/db40b405-17ab-4d10-9e41-545bd4bfa0d6>